Skill progress monitoring

Catching robot failures while they happen

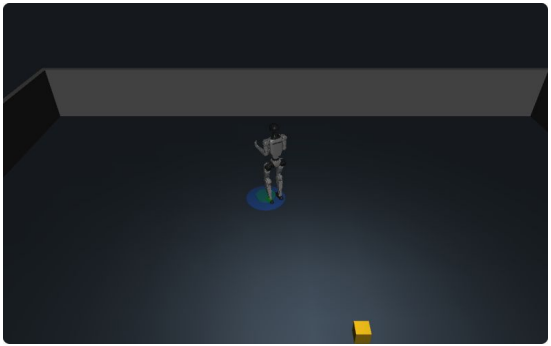
Pinhas Nehoray Aburmad

Ben-Gurion University of the Negev
Supervisor: Prof. Ronen Brafman



The problem

- Robots often fail **in the middle** of a task.
- They often **do not notice**.
- They report “done” anyway.
- **Goal:** notice the failure **the moment it happens**.



The robot says “done” at the drop-off (blue). The box (yellow) is still on the floor.

Related work 1: monitors

Work	What it does / what is missing
[Runtime Verification for LTL and TLTL, Bauer, Leucker, Schallhart, 2011]	Checks LTL rules while the system runs: yes / no / not yet. No task stages: “eventually” stays “not yet” to the end.
[ROSMonitoring: A Runtime Verification Framework for ROS, Ferrando et al., 2020] [RTAMT: Runtime Robustness Monitors with Application to CPS and Robotics, Yamaguchi, Hoxha, Ničković, 2024]	Monitors that run next to a robot and read its messages. Rules written by hand; no idea of task progress.
[Runtime Verification with State Estimation, Stoller et al., 2011]	Guesses a hidden state with a probability model. Needs that model; no task stages.

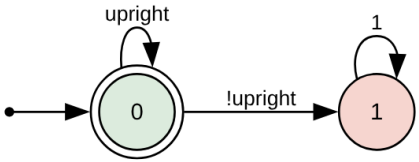
Related work 2: from words, from data

Work	What it does / what is missing
[Grounding Complex Natural Language Commands for Temporal Tasks in Unseen Environments, Liu et al., 2023] [nl2spec: Interactively Translating Unstructured Natural Language to Temporal Logics with LLMs, Cosler et al., 2023] [NL2TL: Transforming Natural Languages to Temporal Logics using LLMs, Chen et al., 2023]	Turn English sentences into LTL rules. Used to plan or to write specs, not to watch a running task.
[FailSafe: Reasoning and Recovery from Failures in VLA Models, Lin et al., 2025] [Foresight: Failure Detection for Long-Horizon Robotic Manipulation, Zhang et al., 2026] [Runtime Monitoring of Perception-Based Autonomous Systems via Embedding Temporal Logic, Kapoor et al., 2026]	Learn to spot failures from camera images. Need training data; hard to say why an alarm fired.

Our gap: rules made from plain words **+** task stages **+** only the robot's own sensors.

Background: what is a monitor?

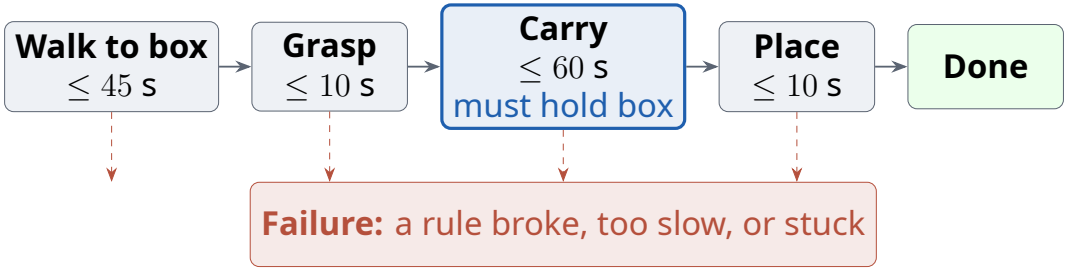
- A program that **watches** the robot's messages.
- It checks **rules over time**, written in **LTL** (a logic of time).
- Example: "*always* stay upright".
- Each rule becomes a small **state machine**.
- Problem: "*eventually* deliver" **never fails early**.



"Always upright" as a state machine. One fall moves it to state 1 (red), forever.

Our idea: the stages of the task

Each stage has its own rules and its own time limit.



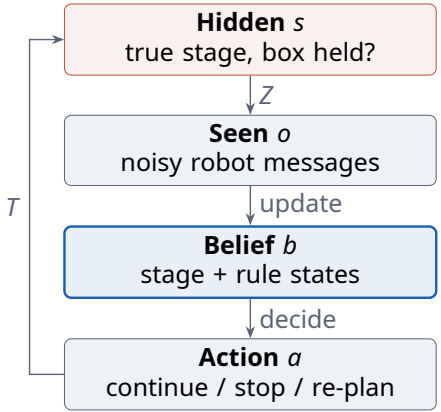
While carrying, the robot **must hold the box**. A drop is caught **at once**.

The mathematical model: a POMDP

- **Hidden** s: what really happens (stage, box held?).
- **Seen** o: the robot's messages.
- **Belief** b: our best guess of s.
- **Action** a: continue, stop, or re-plan.

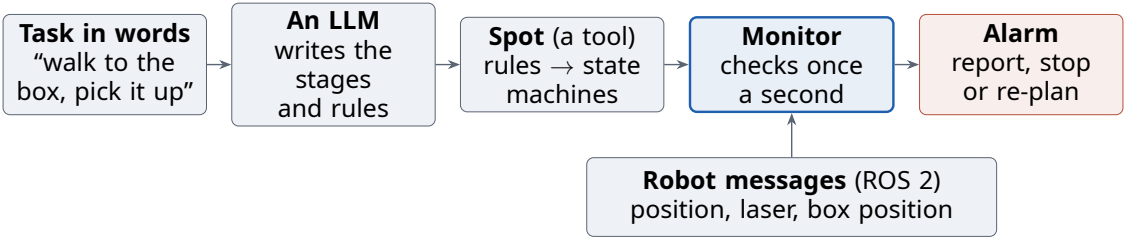
$$b'(s') \;\propto\; \underbrace{Z(o \mid s')}_{\text{does } o \text{ fit } s'?} \sum_{s} \underbrace{T(s' \mid s, a)}_{\text{how } s \text{ changes}} \; b(s)$$

Today: the guess is one point (stage + rule states); the update is one rule step.



[Planning and Acting in Partially Observable Stochastic Domains, Kaelbling, Littman, Cassandra, 1998]

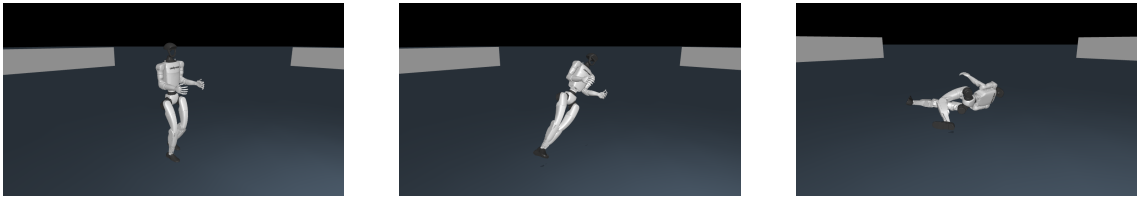
Method: how the monitor is built and run



- The rules are written **automatically**, from the task in words.
- The monitor uses **only sensors**. It never asks the planner.

Test bed: a humanoid robot in a simulator

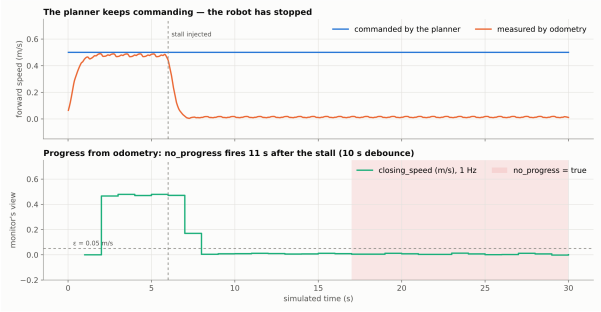
- Unitree **G1** robot in the **MuJoCo** physics simulator.
- We **cause failures on purpose**: push, stall, drop, failed grasp.
- We compare the monitor **with stages** and **without**.



Walking → pushed → fallen. The simulator knows the truth; the monitor does not.

Experiment 1: a short walk (1800 runs)

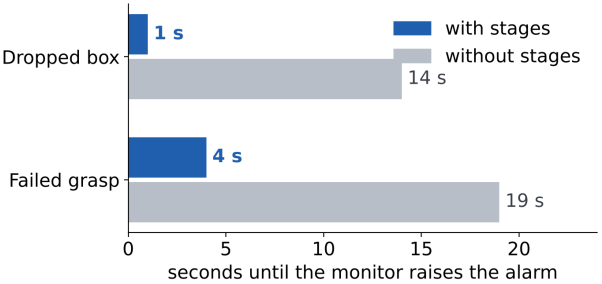
- Stages helped **little** here: 15 vs. 14 of 15 stalls caught.
- What mattered: measure progress **from the robot's sensors**.
- Without that: only **1 of 15** caught.



The planner still says "go". The robot has stopped. The monitor sees no progress.

Experiment 2: fetch and deliver (3360 runs)

- Four stages.
- Box dropped: caught after **1 s** with stages, 14 s without.
- Grasp failed: **4 s** vs. 19 s.
- Earlier in **all 20** runs.

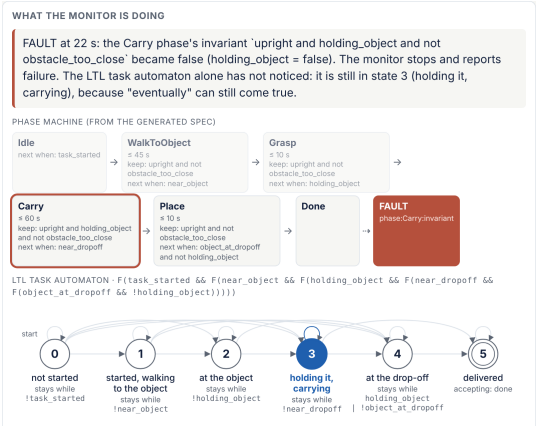


Middle value (median) of 20 runs for each failure.

One run, second by second

- **21.6 s:** the box drops.
- **22 s:** the Carry rule breaks. Alarm.
- The plain LTL rule still waits: “maybe later”.

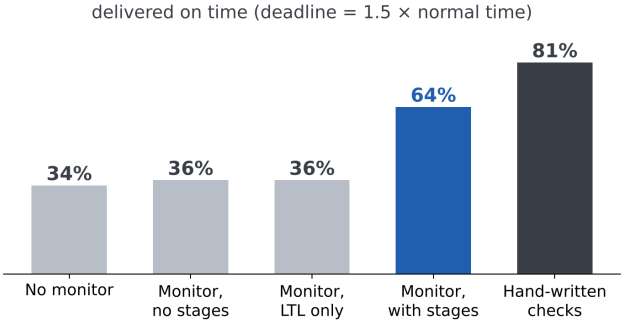
The stage caught it.



From the live demo (one run). The current stage and state are lit.

Does catching early matter?

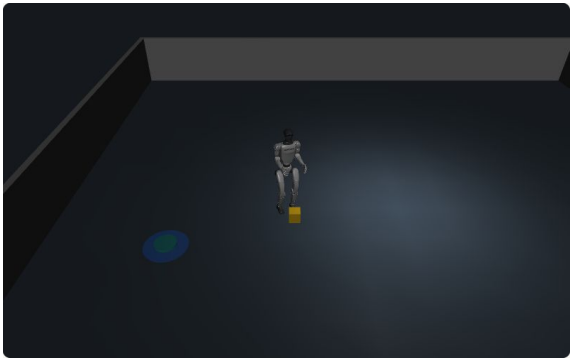
- With no time limit: all monitors ≈ 82–84%.
- With a time limit: **64%** with stages, 36% without.
- Hand-written checks: 81%, but someone must **predict every failure**.



Tasks finished on time, 160 runs per bar. Early knowledge pays when time matters.

Failures nobody planned for

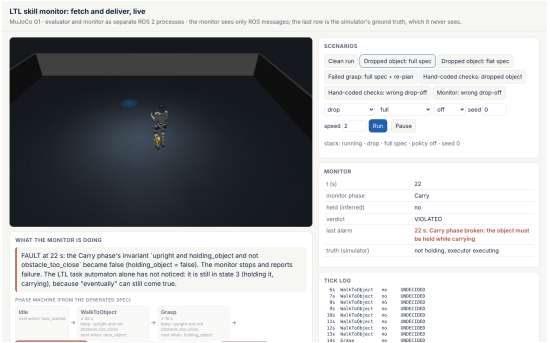
- Box left at the wrong place; robot too close to a wall.
- Hand-written checks: **40 of 40** missed.
- With our monitor: **0 of 40** missed.
- Missed by all: too slow, or a long detour.



The box is far from the drop-off (blue), yet the robot says “done”. Our monitor catches it.

What we learned

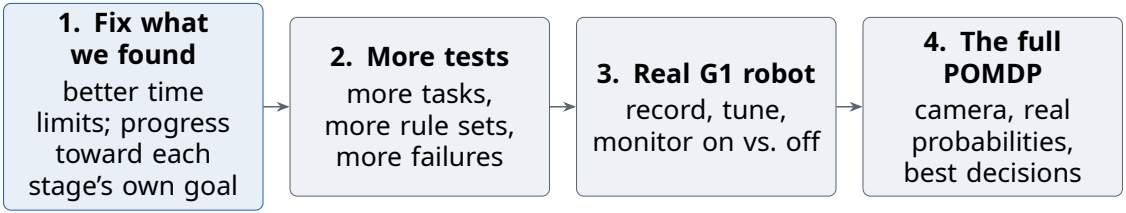
- **Stages catch failures when they happen.**
- Early alarms help **when time matters**.
- Rules come from words, so failures **nobody planned for** are caught too.
- Use the **robot's own sensors**, not the planner.



Live demo, one command:

```
docker run -p 8080:8080
ghcr.io/pinhas019/ltlmonitor-demo
```

To be done



Limits today: the grasp is simulated (no arms on this G1), the camera is simulated, the thresholds are not tuned.

References I

[Runtime Verification for LTL and TLTL, Bauer, Leucker, Schallhart, 2011]. ACM TOSEM 20.

[NL2TL: Transforming Natural Languages to Temporal Logics using Large Language Models, Chen, Gandhi, Zhang, Fan, 2023]. EMNLP.

[nl2spec: Interactively Translating Unstructured Natural Language to Temporal Logics with Large Language Models, Cosler, Hahn, Mendoza, et al., 2023]. CAV.

[ROSMonitoring: A Runtime Verification Framework for ROS, Ferrando, Cardoso, Fisher, Ancona, et al., 2020]. TAROS.

[Planning and Acting in Partially Observable Stochastic Domains, Kaelbling, Littman, Cassandra, 1998]. Artificial Intelligence 101.

[Runtime Monitoring of Perception-Based Autonomous Systems via Embedding Temporal Logic, Kapoor, Hammer, Kapoor, et al., 2026]. arXiv.

[FailSafe: Reasoning and Recovery from Failures in Vision-Language-Action Models, Lin, Duan, Fang, Fox, et al., 2025]. arXiv.

[Grounding Complex Natural Language Commands for Temporal Tasks in Unseen Environments, Liu, Yang, Idrees, Liang, et al., 2023]. CoRL.

[Runtime Verification with State Estimation, Stoller, Bartocci, Seyster, Grosu, Havelund, Smolka, Zadok, 2011]. RV, LNCS 7186.

References II

[RTAMT: Runtime Robustness Monitors with Application to CPS and Robotics, Yamaguchi, Hoxha, Ničković, 2024]. STTT 26.

[Foresight: Failure Detection for Long-Horizon Robotic Manipulation with Action-Conditioned World Model Latents, Zhang, Lu, Wang, Kang, et al., 2026]. arXiv.